RE-call: Evidence-First Memory, for AI Agents

Short version: the AML service and runs

Giulio D'Erme

https://github.com/GiulioDER/RE-call

October 2026

Abstract

RE-call is an open source memory system built to return evidence that can be checked and to say when it has none.

It preserves each source with its provenance and binds every vector to the exact model and pipeline that produced it. Ranking accounts for validity as well as similarity. When nothing clears a calibrated threshold, it returns an abstention with a reason. The same engine holds hand-written notes, documents, source code, conversations and images.

For AML I built `recall_aml`, a service on that engine that returns ranked, dated evidence and never an answer.

Contents

1 The AML service 3

2 Models and runs 5

3 Scope and limits 5

4 Textual 6

5 Coding 7

6 Multimodal 8

7 API, deployment and reproduction 9

8 Attribution and licences 10

1 The AML service

`recall_aml` uses RE-call's store, embedding registry, embedding cache, atomizer and atomic rescue, and adds the AML API, routing and per-user isolation. AML's contract has the platform's reader decide from the returned evidence, so the service always returns ranked evidence and does not use the trust gate's abstention. Through RE-call's MCP server, an agent can also question the memory directly. It can ask for the evidence behind a hit, related memories, the current value of a fact, or why a search abstained, and it can report a memory as stale (Section 5 of the full paper). AML's Add and Search contract measures retrieval alone, so these tools are outside the runs. Figure 1 shows how the service handles an Add and a Search.

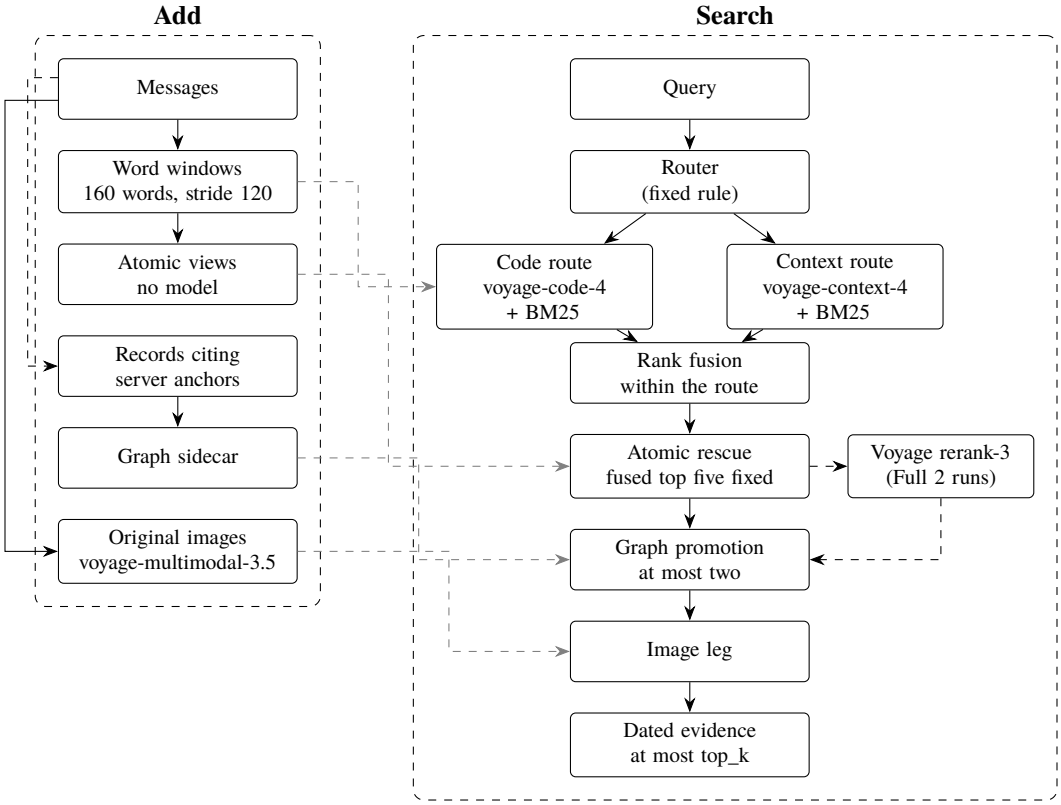


Figure 1: How the C9 variant handles an Add and a Search. Dashed black arrows are optional: records exist only when the compiler is on, and the reranker runs in the second Full runs. Dashed grey arrows show which stored structure each Search stage reads.

Add. The messages of a text Add are joined in order and stored as overlapping windows of 160 words with a stride of 120. Every word is kept, in order; whitespace is reduced to single spaces. The raw windows are never replaced by a summary. From the same request, with no model, a text Add builds its atomic views. A message that carries an image is stored as one record holding its text and image parts in order. Each image is stored once per user as its original bytes. When the compiler is on, gpt-4o-mini writes at most eight typed records per Add. It may cite only anchor identifiers that the server created from the Add's messages. The server builds every quote from those anchors, so a record holds only text that is in a source message. Records feed the graph sidecar.

Search. A fixed rule with no model routes each query. A query that carries an image takes the image route, which searches the code store beside the image index (voyage-multimodal-3.5). Other text goes to context (voyage-context-4) when it asks about conversational memory and names no code, and to code (voyage-code-4) otherwise. In the Textual and Multimodal runs, the image leg also runs on every route for a user whose memory holds images. On the chosen route, exact dense retrieval and BM25 each return 100 candidates, and reciprocal rank fusion merges them. Image hits join by rank, so no score is compared across models. Atomic rescue can lift a window, never into the fused top five; in the Textual and Multimodal runs, rerank-3 then reorders the whole list. Graph promotion can move at most two windows that retrieval already found, never into the first eight. Each returned text item opens with its own date. An item's date comes from the timestamps of its source messages, never from the time the Add arrived. A text window takes the latest timestamp in its Add, and an image-bearing message takes its own. So a replayed history keeps its own timeline. An Add whose messages carry no timestamp gives an undated item.

Isolation. Each `user_id` maps to its own tenant, `aml_` plus the SHA256 of the id, and PostgreSQL row level security applies it inside every pooled transaction. Images, image vectors, specialist vectors, graph records and atomic views live in separate tenants under the same user boundary, and Delete erases all of them in one transaction. An Add is durable before it returns 200, a repeated `request_id` returns the stored response, and one user's writes are applied one at a time.

What the service adds to the engine:

1. Atomic views built inside each Add, so a Search between two Adds already uses them.

2. A graph the model cannot draw: edges come only from verified spans, and promotion only reorders what retrieval found.

3. Dates a reader can use: each item is dated from its messages' timestamps. A relative phrase also carries the date it refers to, and keeps its original words.

4. In the Textual and Multimodal runs, images are searched wherever the user's memory holds them, not only when a question names a visual word.

Why the derived evidence can be trusted. A summary, an extracted fact or a model's paraphrase can introduce text the user never said and later serve it as memory. RE-call constrains both structures derived from an Add to text in its source messages.

- Atomic views are cut from the Add's own stored windows by a fixed rule, with no model, and each keeps a link to its parent window. They exist to be found: a short view matches a narrow question that its long window would not. When one matches, the reader receives the parent window, the user's own words in their context, never the fragment alone. Because the views depend on the request alone, a retried Add rebuilds them byte for byte.

- Compiled records are chosen by the model and written by the server. The server cuts the Add's messages into anchors with identifiers it creates, and the model may only name a kind and the identifiers that support it. The server then builds every span from those anchors. It keeps a field only when its text appears in a cited span, and applies the same check to every named entity, event time and supersession. Anything the model wrote that the sources do not contain is dropped. The graph's edges are drawn from these verified spans, so the graph cannot hold a link the sources do not support either.

2 Models and runs

Embedders: voyage-code-4, voyage-context-4 and voyage-multimodal-3.5 (Voyage AI). Generative model: gpt-4o-mini only (`openai/gpt-4o-mini` through OpenRouter, temperature 0). The Add compiler uses it. When the compiler is on, one readiness check at service start also uses it: the check compiles a fixed test sentence and never sees evaluation data. Search calls no generative model in any run. The Textual and Multimodal runs also reorder text candidates with Voyage rerank-3.

Every run is served by the variant `C9_routed_specialists_grounded_graph_atomic`, with atomic rescue on and every item dated from its source messages.

	Textual	**Coding**	**Multimodal**
Commit	`625b7138`	`1f8666df`	`625b7138`
Compiler	off	on, kind and anchors only	off
Graph promotion	no records to use	on	no records to use
Relative dates	all routes, code text left alone	off on the code route	all routes, code text left alone
Image leg	any user holding images	questions with an image or a visual word	any user holding images
Reranker	rerank-3	none	rerank-3
Feedback guard	yes	no	yes
Run	`teval_ 03cf49b2d9947ba6`	`teval_ 71d752860575cb1b`	`teval_ 9843aa9d742f0fd4`
Result	52.41	59.67	38.60

Each commit carries a signed tag in the repository: `aml-c2-full1-coding` and `aml-c2-full2-625b7138`. `/version` on the endpoint reports the commit as `git_commit`. The product version field reads 1.0.0 on every build, so it does not identify one.

3 Scope and limits

- Two of the five rules in Section 2 of the full paper, abstention and declared supersession, are not exercised by AML's contract and data. The runs exercise evidence, identity, dates and recovery. Section 5 of the full paper describes the supersession design and why the AML service declares no supersession from raw conversation.

- Windows reduce whitespace to single spaces, so the indentation of code is not returned.

- Relative date annotation recognises a fixed list of English phrases; a phrase outside the list, or in another language, is returned unchanged.

- With the compiler off, as in the Textual and Multimodal runs, the graph has no records to use.

- An abstention is returned together with the hits it judged, so the caller can inspect them; RE-call cannot stop an agent that ignores it.

4 Textual

Commit `625b7138`, run `teval_03cf49b2d9947ba6`. Configuration: `RECALL_AML_ROUTE_GATES=data`, `RECALL_AML_T1_GATE=content`, `RECALL_AML_T1_RENDER=v2`, `RECALL_AML_COMPILER=0`, `RECALL_AML_RERANKER=1`, `RECALL_AML_RERANK_MODEL=rerank-3`, `RECALL_AML_FEEDBACK_GUARD=1`; every other setting is the default at the commit.

1. The compiler is off, so no generative model runs during Add, Search or service start, and the graph has no records to use.

2. Voyage rerank-3 reorders the whole fused candidate pool after atomic rescue. A query that is only an image keeps its fused order. A reranker failure or timeout serves the fused order.

3. A relative time phrase keeps its words, and the date it refers to follows it in brackets, computed from the item's own date. Week expressions stay relative to an anchor date. Code-like items and phrases are left as written.

4. Some stored records are earlier answers that report a failure, on a question this user was already asked. Such a record is not kept: published streaming studies find that putting a model's past failed outputs back into its context can lower its accuracy or task success [14, 15]. Two record shapes are recognised, the shapes that the public AgentMemoryBench protocol writes [4]. One is a message that is a JSON object with a question, the model's answer and its outcome. The other is a judge verdict that closes a question and answer turn, and the whole turn is dropped. A failed record that also carries the reference answer is kept. Every other message is stored unchanged.

5. The image leg runs for any user whose memory holds images.

6. Each user's writes are applied one at a time, and Voyage requests stay within the provider's limits.

7. Each Search has a 90 second budget. A rerank or image search that runs out of time is skipped, and a question with text is answered from its text ranking. A query that cannot be embedded in time is answered with HTTP 503 and Retry-After.

8. An image that cannot be read is skipped at Add. The Add is refused only when nothing in it can be read.

Add and Search concurrency are each declared as 16. The platform recorded ten interrupted attempts between 2026-10-25 and 2026-10-28: four `SEARCH_TIMEOUT`, two `PARTICIPANT_ENDPOINT_UNAVAILABLE` and four `EVAL_API_CONTRACT_MISMATCH` (HTTP 400), each resumed from its checkpoint. The service was not restarted during the run. In the part of the run its journal still holds, from 2026-10-26 23:25, every one of the 1,600 Adds and 1,693 Searches it received was answered with HTTP 200.

Result, as the platform reports it: 52.41, completed on 2026-10-28 at 19:01 after 80 hours 9 minutes, a duration that includes the interrupted attempts. The first Textual Full scored 48.92 (Section 15 of the full paper).

5 Coding

Commit `1f8666df`, run `teval_71d752860575cb1b`. Coding questions take the code route, which is built for code:

- Dense retrieval uses voyage-code-4, exact, with no approximate index. BM25 runs beside it over the window text, so an identifier, a path or an error string is found by its own tokens.
- Windows hold message content only, with no role or timestamp mixed into the tokens, and every word of the code is kept in order.
- Ties follow a fixed window order, so the same query over the same memory returns the same list.
- The compiler marks what a session holds (symptom, root cause, failed attempt, successful repair, architectural decision, procedure, validation, constraint, repository fact) and writes only each record's kind and the anchors it cites. The reader receives stored windows of the user's own words, never a paraphrase of the code.
- Graph promotion can lift a window that a compiled record cites.
- Relative date annotation is off on the code route, so a date expression inside code is never rewritten.
- When the compile credit is exhausted, an Add stops with a retryable 503 and stores nothing. So no Add is silently stored without its records.

The compile is bounded. The Add's own anchors are capped at 150,000 characters; a longer Add is compiled from its first and last anchors. The earlier records sent with it are capped at 145,000.

Declared concurrency, as the platform records it: Add 32, Search 64. The platform recorded six interrupted attempts on 2026-09-29. Each was a retryable `SYSTEM_RUNTIME_ERROR` in the platform's system stage, with the message "Coding execution failed", and the run then completed.

Result, as the platform reports it: 59.67, from 300 graded attempts over 150 tasks. Relevant scored 60.67 (91 of 150 resolved, files localized 94.67%), and Noisy 58.67 (88 of 150 resolved, files localized 92.00%).

6 Multimodal

Commit `625b7138`, run `teval_9843aa9d742f0fd4`, labelled "RE-call C9 625b7138 Multimodal Full 2", of the registered version "RE-call C9 625b7138". Its version disclosure is public in the repository as `docs/AML_MULTIMODAL_FULL2_DISCLOSURE.md`, at commit 987a4045. The run has the configuration and behaviour of the Textual run (Section 4, items 1 to 8) and the same declared concurrency. What is specific to images:

1. Each image is stored once per user as its original bytes and embedded with voyage-multimodal-3.5. Search returns an image-bearing message as its ordered text and image parts. The reader receives the exact original image in its place in the message, not a caption.

2. Image search runs for any user whose memory holds images, not only when the question names a visual word. A user without images gets the text ranking unchanged.

3. Text memories are returned beside images on the image route.

4. Image items carry the timestamp of their message, when it has one.

5. Voyage rerank-3 reorders text candidates; image hits are fused after it.

6. An image that only image search found is returned as the image itself, not as placeholder text.

Disclosures. Times are UTC.

1. The platform interrupted the run once. It started on 2026-10-21 at 20:16, and the platform marked it failed on 2026-10-22 at 02:37:44, after 6 hours 21 minutes, with the message "Participant endpoint unavailable". The run was still in its Add phase, and no Search had been sent.

2. The service was healthy throughout. Its process never restarted, and it answered Adds with HTTP 200 until 02:37:51. The tunnel logged no event near that time.

3. The platform restarted the run at about 09:03 on the same deployment. The service was not restarted, and the memory stored before the interruption stayed in its database.

4. Three Adds received a transient HTTP 503 during the run, at 22:35, 01:37 and 02:30, each after a connection error to the embedding provider.

5. The restarted run resumed from where it stopped; it did not start over. The 16 users who had received Adds before the interruption received further Adds after it, and the service's database holds no stored window twice for any of them. Over both attempts the service stored 55,805 Adds for 812 users, every one successfully.

6. Some Adds took longer than the tunnel's limit of about 100 seconds, so the platform received a timeout and sent them again. An Add is idempotent by its request id, so none was stored twice.

7. The Search phase ran from 2026-10-24 01:54 to 20:45. Every one of the 5,165 Search requests was answered with HTTP 200, the slowest in 18 seconds, inside the 90 second budget. On one Search the reranker failed and the fused order was served. On one Search the image search ran out of time and the text ranking was served.

8. Two requests to the embedding provider lost their connection and succeeded on retry. On eleven occasions a write to the service's local embedding cache was skipped because the cache was busy; the vector was still computed and stored, so no result changed.

9. The service's journal for the first part of the run rotated before the run ended. The Add counts above come from the service's database, which holds every Add of both attempts; the Search counts come from the journal, which holds the whole Search phase.

Result, as the platform reports it: 38.60 over 4,943 questions, completed on 2026-10-24 at 22:35 after 74 hours 19 minutes, a duration that includes the interrupted attempt and the wait before the restart. The first Multimodal Full scored 31.64 over the same 4,943 questions (Section 15 of the full paper).

Retrieval	Relational reasoning	Temporal reasoning	Continual learning	Personalization	Evidence governance
46.43	38.94	31.96	35.02	8.20	26.53

Subcapability	First Full	Second Full
direct_recall	23.60	32.94
information_extraction	22.13	46.31
atomic_retrieval	52.18	66.28
visual_reasoning	19.35	32.40
multi_hop_reasoning	25.83	29.67
multi_session_reasoning	10.00	9.29
relational_association	42.44	46.57
temporal_reasoning	28.43	31.96
knowledge_update	20.17	23.28
evolutionary_synthesis	52.29	58.54
implicit_preference	9.84	8.20
abstention	1.57	2.75
answer_refusal	95.45	95.45

Most of the gain is in retrieval, where the first run had two served defects (Section 15 of the full paper). Because the runs also differ in variant, compiler, reranker, dates and feedback rule, the results cannot isolate each change's contribution. Abstention stays near zero: under AML's contract, the service returns evidence and never declines (Section 3).

7 API, deployment and reproduction

Endpoint: the registered endpoint, self-hosted and served over HTTPS through a Cloudflare Tunnel: `POST /v1/add`, `POST /v1/search`, `POST /v1/delete`, `GET /health`, `GET /version`. Requests authenticate with the dedicated evaluation credential, sent as `Authorization: Bearer <key>` or `X-Api-Key: <key>`. The endpoint stays reachable, with its API contract, authentication and capacity unchanged, until 2026-11-04 23:59 UTC+8.

Capacity: one service process on one Linux server, with a 12 GB memory limit in the Full 2 runs. Declared concurrency, as the platform records it per run: Coding Full 1 Add 32 and Search 64; Textual Full 1 Add 32 and Search 128; Multimodal Full 1 Add 16 and Search 16; both Full 2 runs Add 16 and Search 16. A request beyond the service's own concurrency waits for a free slot; none is refused for load. HTTP 422 is returned only for input that can never be accepted, such as invalid JSON, a body over 44 MiB, a missing `request_id`, `user_id` or `session_id`, a malformed image part, or an Add in which nothing can be read. A failure inside the service returns a retryable 503. So does a Search whose query cannot be embedded within its 90 second budget, with Retry-After.

Observed in Multimodal Full 2: the service ran as one process for the whole run, with no restart, and its memory peaked at 2.07 GB of the 12 GB limit. Over the last 11 hours of Adds, an Add took a median of 20.3 seconds and at the 99th percentile 76.6 seconds. 36 took longer than the tunnel's limit of about 100 seconds, so the platform received HTTP 524 and sent them again; none was stored twice. A Search took a median of 2.1 seconds and at most 18.1 seconds. After the run, on 2026-10-25 at 08:56 UTC, the service was restarted once on the same build, before Textual Full 2.

Run environment, read from the live service on 2026-10-21 for the Textual and Multimodal runs: Python 3.12.3, PostgreSQL 17.11 with pgvector 0.8.7, schema version 0024. `GET /version` reports the commit, the variant and every active component, so a reviewer can check the running build against this document. The service starts as `python -m recall_aml` from an install of the repository at the fixed commit, with:

- `RECALL_AML_VARIANT=C9_routed_specialists_grounded_graph_atomic`
- `RECALL_AML_DATABASE_URL`, `RECALL_AML_API_KEY`, `RECALL_AML_GIT_COMMIT` (the fixed commit)
- `VOYAGE_API_KEY`, and `OPENROUTER_API_KEY` when the compiler is on
- for the Textual and Multimodal runs, the seven settings listed in Section 4.

8 Attribution and licences

RE-call is my own work. The code that serves the AML runs is mine; it uses these components unchanged:

- PostgreSQL (PostgreSQL License) and pgvector (PostgreSQL License), for storage and vector search.
- BM25 [2] and reciprocal rank fusion [3], implemented in the repository.
- Voyage AI models voyage-code-4, voyage-context-4, voyage-multimodal-3.5 and rerank-3, used through Voyage AI's API under its terms of service.
- The tokenizers of voyage-context-4 (Hugging Face `MongoDB/voyage-context-4`, Apache-2.0) and voyage-multimodal-3.5 (Hugging Face `MongoDB/voyage-multimodal-3.5`, no licence declared on its page), downloaded at run time only to keep a request within the model's input limit.
- OpenAI gpt-4o-mini, used through OpenRouter under their terms of service.
- Python libraries, all unmodified: psycopg (LGPL-3.0), certifi (MPL-2.0), tqdm (MPL-2.0 and MIT), and permissively licensed libraries including pydantic (MIT), Starlette (BSD-3-Clause), the `voyageai` and `openai` clients and numpy. The complete pinned list is `uv.lock` at each commit.

Two texts in the repository are adapted from other projects, and neither serves the AML runs:

- The "dated" reading instructions of the MCP reasoning path (`recall/evidence.py`) are adapted from the reader contract of vectorize-io/hindsight (MIT).

- Research harnesses under `benchmarks/`, which are not shipped in the package, carry evaluation prompts copied verbatim, so that a comparison uses the benchmark's own text. These are the BEAM prompts of mem0ai/memory-benchmarks (Apache-2.0, recorded in `benchmarks/beam/UPSTREAM.md`), and the answer and judge prompts of AML's own public harness, the benchd harness and the MTRAG benchmark.

Licence of RE-call: AGPL-3.0-only. Every fixed commit named in this document carries the PolyForm Noncommercial License 1.0.0 in its tree. `LICENSE_GRANT_AGPL.md` in the repository licenses each of them under AGPL-3.0-only, by full commit hash.

References

[1] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks", Advances in Neural Information Processing Systems 30 (NeurIPS 2017). https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

[2] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond", Foundations and Trends in Information Retrieval 3(4), pp. 333 to 389, 2009. https://doi.org/10.1561/1500000019 Cited on page 10.

[3] G. V. Cormack, C. L. A. Clarke and S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods", SIGIR 2009, pp. 758 to 759. https://doi.org/10.1145/1571941.1572114 Cited on page 10.

[4] AgentMemoryBench, "Benchmarking Continual Agent Memory for Online Learning, Transfer, and Forgetting", MIT licence. The feedback rule recognises the record shapes of its protocol, read at commit `9dfda8de`. It is unrelated to my own benchmark harness of a similar name. https://github.com/solomoon313/AgentMemoryBench/tree/9dfda8dede15bf16518524e4614ed89344795413 Cited on page 6.

[5] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang and D. Yu, "Dense X Retrieval: What Retrieval Granularity Should We Use?", EMNLP 2024, pp. 15159 to 15177, arXiv:2312.06648. https://doi.org/10.18653/v1/2024.emnlp-main.845

[6] T. Gao, H. Yen, J. Yu and D. Chen, "Enabling Large Language Models to Generate Text with Citations", EMNLP 2023, pp. 6465 to 6488. https://doi.org/10.18653/v1/2023.emnlp-main.398

[7] J. Strötgen and M. Gertz, "HeidelTime: High Quality Rule-Based Extraction and Normalization of Temporal Expressions", Proceedings of the 5th International Workshop on Semantic Evaluation (SemEval 2010), pp. 321 to 324. https://aclanthology.org/S10-1071/

[8] A. X. Chang and C. D. Manning, "SUTime: A library for recognizing and normalizing time expressions", LREC 2012, pp. 3735 to 3740. https://aclanthology.org/L12-1122/

[9] H. Lee, L. Soldaini, A. Cohan, M. Seo and K. Lo, "RouterRetriever: Routing over a Mixture of Expert Embedding Models", AAAI 2025, pp. 11995 to 12003. https://doi.org/10.1609/aaai.v39i11.33306

[10] R. Nogueira and K. Cho, "Passage Re-ranking with BERT", arXiv:1901.04085, 2019. https://arxiv.org/abs/1901.04085

[11] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", arXiv:2404.16130, 2024. https://arxiv.org/abs/2404.16130

[12] P. Rajpurkar, R. Jia and P. Liang, "Know What You Don't Know: Unanswerable Questions for SQuAD", ACL 2018 (Volume 2: Short Papers), pp. 784 to 789. https://doi.org/10.18653/v1/P18-2124

[13] A. Kamath, R. Jia and P. Liang, "Selective Question Answering under Domain Shift", ACL 2020, pp. 5684 to 5696. https://doi.org/10.18653/v1/2020.acl-main.503

[14] C.-K. Wu, Z. R. Tam, C.-Y. Lin, Y.-N. Chen and H.-y. Lee, "StreamBench: Towards Benchmarking Continuous Improvement of Language Agents", Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, pp. 107039 to 107063. https://doi.org/10.52202/079017-3398 Cited on page 6.

[15] T. Wei et al., "Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory", arXiv:2511.20857v2, 2025. https://arxiv.org/abs/2511.20857v2 Cited on page 6.

[16] H. Deng and P. Sinchaisri, "Stale, Misattributed, or Late: Where Personal Memory Fails Before Generation", arXiv:2610.10265, 2026. https://arxiv.org/abs/2610.10265

[17] V. Reddy and S. R. Challaram, "Reliable Post-Retrieval Assembly for Agent Memory: Separating Evidence Extraction from Policy Execution", Lifelong Agent Workshop at COLM 2026, arXiv:2606.01435v2. https://arxiv.org/abs/2606.01435v2